

EKS Production Patterns

Detailed guide with deployment diagrams, step-by-step explanations and configuration code

Key Difference in One Line

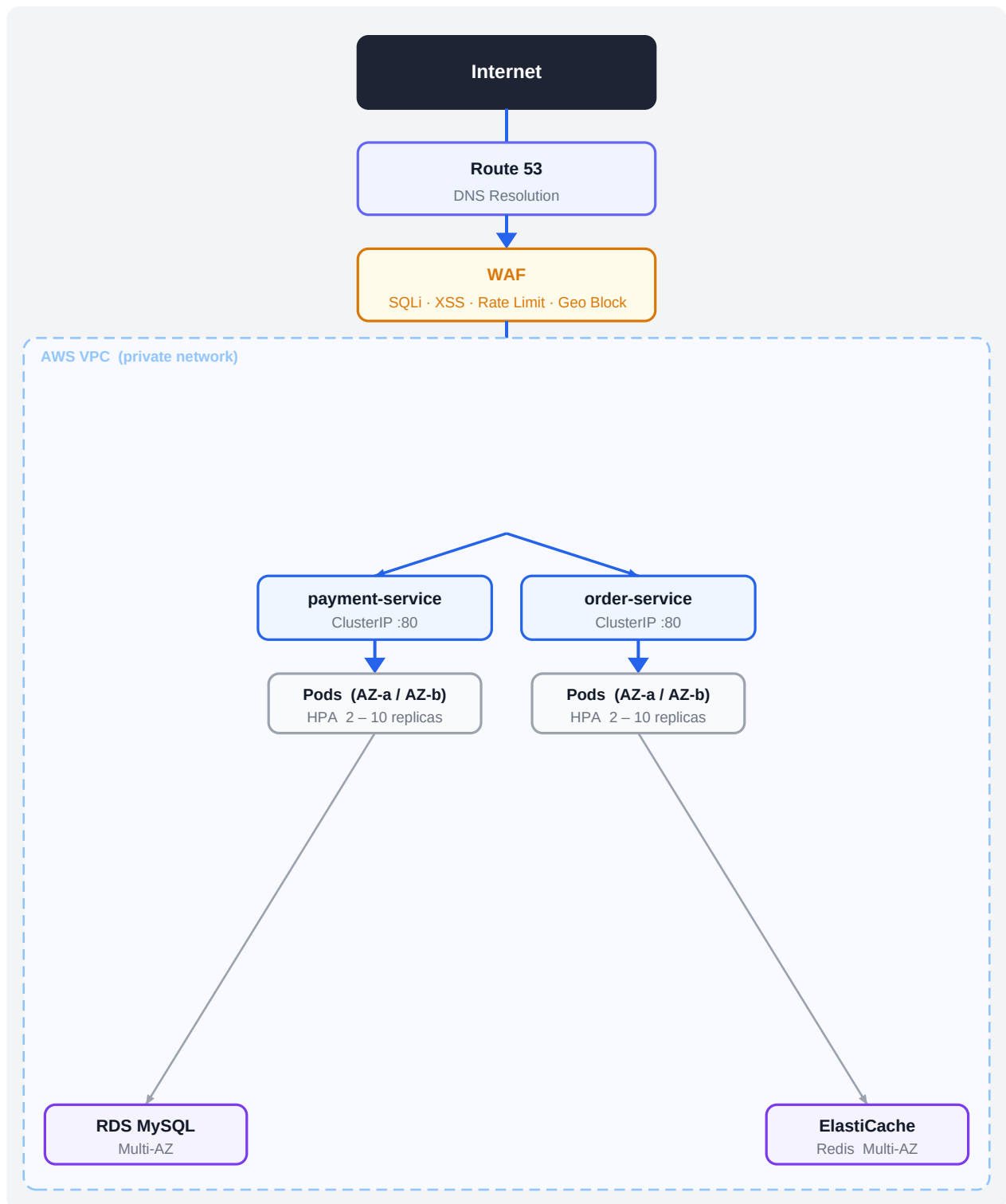
Pattern 1 — ALB is your front door, WAF is your bouncer

Pattern 2 — API Gateway is your front door, ALB is hidden inside your VPC

Pattern 1 — WAF + ALB (internet-facing)

```
Route53 -> WAF -> ALB (internet-facing) -> Ingress -> EKS Services
```

Deployment Diagram



Pattern 1 — Step-by-Step

Step
1

Tag Public Subnets

Why do we need this?

The AWS Load Balancer Controller needs to know which subnets to place the ALB in. Without these tags the controller cannot discover the subnets automatically and ALB creation will fail.

AWS CLI / kubectl / YAML

```
aws ec2 create-tags \

--resources subnet-pub-1a subnet-pub-1b \

--tags \

Key=kubernetes.io/role/elb,Value=1 \

Key=kubernetes.io/cluster/my-cluster,Value=shared
```

Step
2

Create ACM Certificate

Why do we need this?

HTTPS is non-negotiable for payment APIs. ACM provides free SSL/TLS certificates. The certificate ARN goes into the Ingress annotation so the ALB terminates HTTPS at the load-balancer level before traffic reaches pods.

AWS CLI / kubectl / YAML

```
aws acm request-certificate \

--domain-name api.myapp.com \

--validation-method DNS --region ap-south-1

# Validate via Route53 DNS then note the certificate ARN
```

Create WAF Web ACL

Why do we need this?

WAF inspects every HTTP request before it reaches the ALB. You create WAF first so its ARN is ready to paste into the Ingress annotation. The controller then attaches WAF at ALB creation time — zero unprotected window.

AWS CLI / kubectl / YAML

```
aws wafv2 create-web-acl --name payment-waf --scope REGIONAL \

--default-action Allow={} --rules file://waf-rules.json \

--visibility-config \

SampledRequestsEnabled=true,CloudWatchMetricsEnabled=true,MetricName=payment-waf

WAF_ARN=$(aws wafv2 list-web-acls --scope REGIONAL \

--query 'WebACLs[?Name==`payment-waf`].ARN' --output text)
```

Install ALB Controller (Helm)

Why do we need this?

The ALB Controller is the brain behind Ingress. Without it, an Ingress YAML does nothing in AWS. It watches Ingress resources and automatically creates ALBs, target groups, listeners, and routing rules.

AWS CLI / kubectl / YAML

```
helm repo add eks https://aws.github.io/eks-charts && helm repo update

helm install aws-load-balancer-controller eks/aws-load-balancer-controller \

--namespace kube-system --set clusterName=my-cluster \

--set serviceAccount.create=false \

--set serviceAccount.name=aws-load-balancer-controller
```

Apply Deployment and Service

Why do we need this?

The Deployment runs Spring Boot pods and ensures the desired replica count. The Service (ClusterIP) gives pods a stable internal DNS name that Ingress uses as its backend. Without the Service, Ingress has nowhere to route traffic.

AWS CLI / kubectl / YAML

```
kubectl apply -f deployment.yaml

kubectl apply -f service.yaml # type: ClusterIP

kubectl get pods -n payment-ns # verify running
```

Apply Ingress — Public ALB + WAF + HTTPS

Why do we need this?

The annotation 'scheme: internet-facing' creates a PUBLIC ALB in public subnets. The certificate-arn enables HTTPS on port 443. The wafv2-acl-arn attaches WAF at ALB creation time — no unprotected window. The controller wires everything in AWS automatically from this single YAML.

AWS CLI / kubectl / YAML

```
# ingress-public.yaml (key annotations shown)

annotations:

kubernetes.io/ingress.class: alb

alb.ingress.kubernetes.io/scheme: internet-facing # PUBLIC ALB

alb.ingress.kubernetes.io/target-type: instance

alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:...:certificate/xxx

alb.ingress.kubernetes.io/listen-ports: '[{"HTTP":80},{"HTTPS":443}]'

alb.ingress.kubernetes.io/ssl-redirect: '443' # force HTTPS

alb.ingress.kubernetes.io/wafv2-acl-arn: arn:aws:wafv2:...:webacl/payment-waf/xxx

alb.ingress.kubernetes.io/healthcheck-path: /actuator/health

kubectl apply -f ingress-public.yaml

kubectl get ingress -n payment-ns -w # wait for ALB DNS
```

Create Route53 CNAME -> ALB DNS

Why do we need this?

The ALB gets an auto-generated AWS DNS name. Route53 maps your clean domain (api.myapp.com) to that ALB DNS so the frontend always uses one stable URL, even if the ALB is recreated.

AWS CLI / kubectl / YAML

```
aws route53 change-resource-record-sets --hosted-zone-id \  
  
--change-batch '{"Changes":[{"Action":"CREATE",  
  
"ResourceRecordSet":{"Name":"api.myapp.com",  
  
"Type":"CNAME", "TTL":300,  
  
"ResourceRecords":[{"Value":"k8s-xxx.ap-south-1.elb.amazonaws.com"}]}]}'
```

Test — Verify ALB is Public and WAF is Active

Why do we need this?

Always smoke-test before going live. Confirm HTTPS works, WAF blocks malicious requests, and the Spring Boot Actuator health endpoint responds.

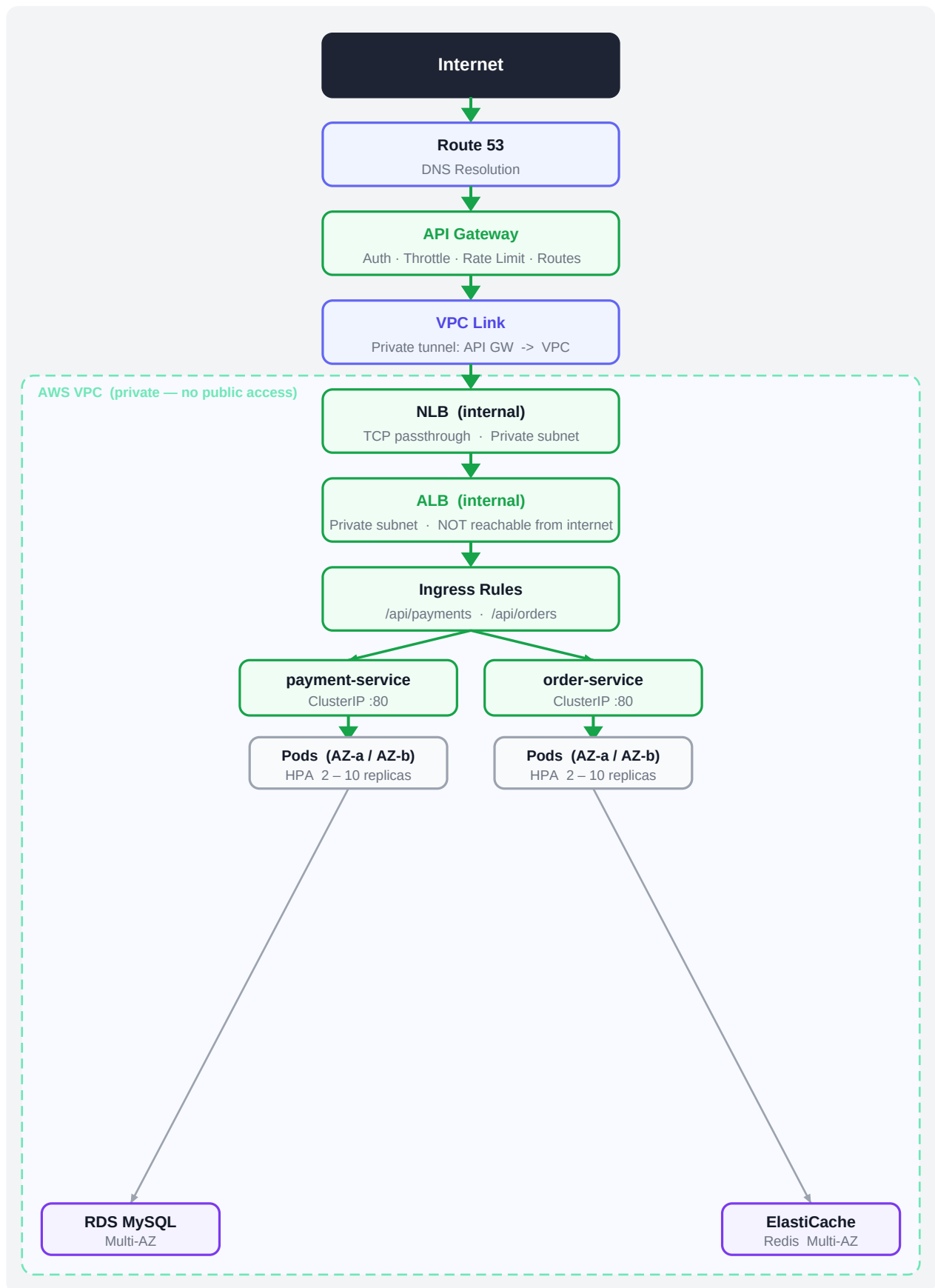
AWS CLI / kubectl / YAML

```
curl https://api.myapp.com/api/payments/health # expect 200 OK  
  
curl 'https://api.myapp.com/api/payments?id=1+OR+1=1' # expect 403  
  
aws elbv2 describe-load-balancers \  
  
--query 'LoadBalancers[?contains(DNSName,`k8s-xxx`)].Scheme'  
  
# Output: ["internet-facing"]
```

Pattern 2 — API Gateway + VPC Link + NLB + ALB (internal)

```
Route53 -> API Gateway -> VPC Link -> NLB -> ALB (internal) -> Ingress -> EKS Services
```

Deployment Diagram



Pattern 2 — Step-by-Step

Step 1

Tag Private Subnets

Why do we need this?

The ALB must be completely hidden inside the VPC. Tagging private subnets with 'internal-elb' tells the ALB controller to place the ALB there, ensuring it only gets a private IP address — never reachable from the internet.

AWS CLI / kubectl / YAML

```
aws ec2 create-tags \

--resources subnet-priv-1a subnet-priv-1b \

--tags \

Key=kubernetes.io/role/internal-elb,Value=1 \

Key=kubernetes.io/cluster/my-cluster,Value=shared
```

Step 2

Create ACM Certificate

Why do we need this?

Even though the ALB is internal, traffic between API Gateway and the ALB still travels over HTTPS. The same certificate is also attached to the API Gateway custom domain for end-to-end encryption.

AWS CLI / kubectl / YAML

```
aws acm request-certificate \

--domain-name api.myapp.com \

--validation-method DNS --region ap-south-1

# arn:aws:acm:ap-south-1::certificate/xxx
```

Install ALB Controller (Helm)

Why do we need this?

Same as Pattern 1. The controller watches for Ingress resources and creates the ALB in AWS. Without it, no ALB is created regardless of the YAML.

AWS CLI / kubectl / YAML

```
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
--namespace kube-system --set clusterName=my-cluster \
--set serviceAccount.create=false \
--set serviceAccount.name=aws-load-balancer-controller
```

Apply Deployment and Service

Why do we need this?

Spring Boot pods must be running before Ingress routes traffic. The Service (ClusterIP) provides the stable internal endpoint that Ingress uses as its backend.

AWS CLI / kubectl / YAML

```
kubectl apply -f deployment.yaml

kubectl apply -f service.yaml # type: ClusterIP
```

Apply Ingress — INTERNAL ALB

Why do we need this?

The annotation 'scheme: internal' is the single most important difference from Pattern 1. The controller creates an ALB with only a private IP in your private subnets. It is physically impossible to reach from the internet. API Gateway via VPC Link is the ONLY path in.

AWS CLI / kubectl / YAML

```
# ingress-internal.yaml (key annotations shown)

annotations:

kubernetes.io/ingress.class: alb

alb.ingress.kubernetes.io/scheme: internal # PRIVATE ALB

alb.ingress.kubernetes.io/target-type: instance

alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:::certificate/xxx

alb.ingress.kubernetes.io/listen-ports: '[{"HTTPS":443}]'

alb.ingress.kubernetes.io/healthcheck-path: /actuator/health

# No WAF – API Gateway is the public entry point

kubectl apply -f ingress-internal.yaml

kubectl get ingress -n payment-ns # note internal ALB DNS

ALB_ARN=$(aws elbv2 describe-load-balancers \

--query 'LoadBalancers[?contains(DNSName,`internal-k8s`)].LoadBalancerArn' \

--output text)
```

Create NLB pointing to Internal ALB

Why do we need this?

VPC Link only supports NLB, not ALB. So an NLB sits in front of the ALB as a TCP passthrough layer. The NLB target type is 'alb', meaning the NLB forwards directly to the internal ALB. The ALB still does HTTP-level routing.

AWS CLI / kubectl / YAML

```
aws elbv2 create-target-group --name payment-nlb-tg \  
  
--protocol TCP --port 443 --vpc-id --target-type alb  
  
aws elbv2 register-targets --target-group-arn \  
  
--targets Id=$ALB_ARN  
  
aws elbv2 create-load-balancer --name payment-nlb \  
  
--type network --scheme internal \  
  
--subnets subnet-priv-1a subnet-priv-1b  
  
aws elbv2 create-listener --load-balancer-arn \  
  
--protocol TCP --port 443 \  
  
--default-actions Type=forward,TargetGroupArn=  
  
# Note NLB listener ARN for API Gateway integration
```

Create VPC Link

Why do we need this?

API Gateway is a fully managed AWS service living outside your VPC. VPC Link is the private network tunnel that lets API Gateway reach into your VPC to talk to the NLB — without exposing anything publicly. Without VPC Link, API Gateway cannot reach your internal NLB at all.

AWS CLI / kubectl / YAML

```
aws apigatewayv2 create-vpc-link \  
  
--name payment-vpc-link \  
  
--subnet-ids subnet-priv-1a subnet-priv-1b \  
  
--security-group-ids  
  
aws apigatewayv2 get-vpc-links # wait for AVAILABLE status  
  
# Note the VPC Link ID
```

Create API Gateway (HTTP API)

Why do we need this?

API Gateway is the only public entry point in Pattern 2. It provides auth, throttling, rate limiting per API key, usage plans, and a developer portal — features ALB does not offer natively. The integration points at the NLB listener ARN via VPC Link so requests flow: API GW -> VPC Link -> NLB -> ALB -> pods.

AWS CLI / kubectl / YAML

```
aws apigatewayv2 create-api --name payment-api --protocol-type HTTP

aws apigatewayv2 create-integration --api-id \

--integration-type HTTP_PROXY --integration-method ANY \

--connection-type VPC_LINK --connection-id \

--integration-uri --payload-format-version 1.0

aws apigatewayv2 create-route --api-id \

--route-key 'ANY /api/payments/{proxy+}' \

--target integrations/

aws apigatewayv2 create-stage --api-id \

--stage-name prod --auto-deploy
```

Create Custom Domain on API Gateway

Why do we need this?

Without a custom domain the API URL would be the ugly auto-generated API Gateway URL. A custom domain gives you `api.myapp.com` and lets you swap the underlying API Gateway without changing what clients call.

AWS CLI / kubectl / YAML

```
aws apigatewayv2 create-domain-name --domain-name api.myapp.com \
--domain-name-configurations CertificateArn=arn:aws:acm::.:certificate/xxx

aws apigatewayv2 create-api-mapping \
--domain-name api.myapp.com --api-id --stage prod

aws apigatewayv2 get-domain-name --domain-name api.myapp.com

# Note ApiGatewayDomainName for Route53
```

Create Route53 CNAME -> API Gateway Domain

Why do we need this?

Route53 points your domain to the API Gateway domain — NOT the ALB. The internal ALB DNS is never exposed publicly. All traffic enters only through API Gateway. This is the key routing difference vs Pattern 1.

AWS CLI / kubectl / YAML

```
aws route53 change-resource-record-sets --hosted-zone-id \  
  
--change-batch '{"Changes":[{"Action":"CREATE",  
  
"ResourceRecordSet":{"Name":"api.myapp.com",  
  
"Type":"CNAME", "TTL":300,  
  
"ResourceRecords":[{"Value":"xxx.execute-api.ap-south-1.amazonaws.com"}]}]}'  
  
# Route53 -> API Gateway domain (NOT ALB DNS)
```

Test — API Gateway Works and ALB is Unreachable

Why do we need this?

The most important test in Pattern 2 is confirming the internal ALB is truly unreachable from the internet. If someone finds the internal ALB DNS and hits it directly, your security model is broken. 'Connection refused' proves it.

AWS CLI / kubectl / YAML

```
curl https://api.myapp.com/api/payments/health # expect 200 OK

curl https://internal-k8s-xxx.ap-south-1.elb.amazonaws.com/api/payments

# Expected: Could not resolve host OR connection refused

aws elbv2 describe-load-balancers \

--query 'LoadBalancers[?contains(DNSName,`internal-k8s`)].Scheme'

# Output: ["internal"]
```

Ingress Annotation Comparison

```
# ■■■ PATTERN 1 – internet-facing (PUBLIC ALB) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

annotations:

alb.ingress.kubernetes.io/scheme: internet-facing # PUBLIC subnet

alb.ingress.kubernetes.io/listen-ports: '[{"HTTP":80},{"HTTPS":443}]'

alb.ingress.kubernetes.io/ssl-redirect: '443' # redirect HTTP to HTTPS

alb.ingress.kubernetes.io/wafv2-acl-arn: arn:aws:wafv2:::webacl/payment-waf/xxx

alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:::certificate/xxx


# Result: PUBLIC IP. Accessible from internet. WAF attached. HTTPS enforced.

# Route53 -> ALB DNS directly.


# ■■■ PATTERN 2 – internal (PRIVATE ALB) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

annotations:

alb.ingress.kubernetes.io/scheme: internal # PRIVATE subnet

alb.ingress.kubernetes.io/listen-ports: '[{"HTTPS":443}]'

alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:::certificate/xxx


# No WAF annotation – API Gateway is the public entry point

# No ssl-redirect – only HTTPS port exposed


# Result: PRIVATE IP only. Not reachable from internet.

# Only reachable via VPC Link from API Gateway.

# Route53 -> API Gateway domain (NOT ALB DNS).
```

	Pattern 1 – WAF + ALB	Pattern 2 – API GW + NLB + ALB
ALB scheme	internet-facing	internal
Subnet	public	private
WAF	attached to ALB	optional on API GW
Route53 points to	ALB DNS	API Gateway domain
Direct ALB access	Yes (WAF protects)	No — truly private
Auth / throttling	in-cluster (Kong etc.)	API Gateway native
Complexity	Simple — 8 steps	Complex — 11 steps
Cost	Lower	Higher
Best for	Microservices / internal	External / partner APIs

Pattern 1 is simpler and cheaper — right for most microservice teams. Pattern 2 gives maximum security and API management features — right for external or partner APIs needing per-client throttling, API keys, and a developer portal. The single annotation 'scheme: internet-facing vs internal' is the line that separates the two architectures.